Part 3

APRENDE PYTHON

✦ Funciones ✦



APRENDE PYTHON

Funciones

Hasta ahora escribíamos código suelto. Una **función** lo empaqueta en una **orden con nombre** que puedes usar una y otra vez, sin copiar y pegar.

```
def saludar():
    print("¡Hola!")

saludar()   # la usas cuando quieras
```

💡 Ya has usado funciones sin saberlo: `print()` y `type()` son funciones que Python ya trae. Ahora crearás **las tuyas**.

def · parámetros · return · reutilizar

Milagros Espinoza Cunya

APRENDE PYTHON
Funciones

1 Crear una función

¿Qué es una función?
Es como una **receta**: la escribes **una vez** y la usas cuando quieras. Se crea con la palabra **def**.

Problema
Mostramos el mismo saludo en varios lugares y no queremos repetir el texto cada vez.

La Solución:
La definimos una sola vez con **def** y luego la **llamamos** por su nombre.

```
def saludar():
    print("¡Hola! Bienvenida a Python")

saludar()
```

`def` → crea la función | `saludar` → el nombre | `saludar()` → la ejecuta

Resultado:

```
¡Hola! Bienvenida a Python
```

DATAZO
Definir con **def** no ejecuta nada todavía: solo "guarda" la receta. El código corre cuando la **llamas** por su nombre con **()**.

Milagros Espinoza Cunya

APRENDE PYTHON
Funciones

2 Parámetros

¿Qué es un parámetro?
Es un **dato que la función recibe** para trabajar. Va dentro de los **()** y lo usas adentro.

Problema
Queremos saludar por su nombre a cada persona, no un saludo genérico.

La Solución:
Ponemos un **parámetro** en los **()** y lo usamos dentro de la función.

```
def saludar(nombre):
    print("¡Hola", nombre + "!")

saludar("Ana")
```

`nombre` → parámetro (el hueco) `"Ana"` → argumento (el dato real)

Resultado:

```
¡Hola Ana!
```

DATAZO

Parámetro = el nombre del hueco cuando defines. **Argumento** = el valor real que envías al llamar. Puedes tener varios, separados por comas.

Milagros Espinoza Cunya

APRENDE PYTHON
Funciones

3 return

¿Qué hace return?
Hace que la función **devuelva un resultado** para guardarlo o seguir usándolo — no solo mostrarlo en pantalla.

Problema
No queremos solo imprimir el promedio: queremos **guardarlo** para seguir calculando.

La Solución:
Usamos **return** para que la función **entregue** el valor.

```
def promedio(a, b):
    return (a + b) / 2

resultado = promedio(8.9, 9.8)
```

Resultado:

```
>>> resultado
9.35   # quedó guardado en la variable
```

DATAZO

Diferencia clave: **print()** solo MUESTRA en pantalla; **return** DEVUELVE el valor para reutilizarlo. En análisis de datos casi siempre usarás **return**.

Milagros Espinoza Cunya

APRENDE PYTHON
Funciones

4 Aplícalo a tu data

El combo poderoso

Una **función** + un **for** = procesar toda tu lista con una sola regla, escrita en un solo lugar.

La Solución:

Creamos una función que clasifica una novela y la aplicamos a todas con un **for**.

```
def clasificar(novela):
    if novela["rating"] > 9.0:
        return "Éxito"
    return "Regular"

for n in novelas:
    print(n["titulo"], "->", clasificar(n))
```

Resultado:

```
Rubí -> Regular
Teresa -> Regular
Rebelde -> Éxito
```

DATAZO

Si la regla cambia, la editas en **un solo lugar** (la función) y todo tu código se actualiza. Así de poderosas son.

5 Funciones que ya existen

No reinventes la rueda

Python trae funciones listas para usar. Ya conoces **print()** y **type()**; estas te ahorrarán mucho trabajo.

```
ratings = [8.9, 9.0, 9.8]
```

`len()`	**Cuántos** elementos hay.	3
`max()`	**El mayor** de todos.	9.8
`min()`	**El menor** de todos.	8.9
`sum()`	**La suma** total.	27.7

Para el promedio: `sum(ratings) / len(ratings)` → combinas dos funciones que ya existen.

DATAZO

Antes de escribir tu propia función, revisa si Python ya la tiene. Casi siempre la respuesta es sí.

APRENDE PYTHON
Funciones

Todo lo que necesitas saber de funciones:

Concepto	Sintaxis	¿Para qué?
Definir	`def nombre():`	Crea la función (la guarda).
Llamar	`nombre()`	La ejecuta cuando tú quieras.
Parámetro	`def f(x):`	Recibe un dato para trabajar.
Devolver	`return`	Entrega un resultado para reutilizar.
Ya incluidas	`len() sum()`	Funciones listas de Python.

¡Guarda este post, dale like, compártelo y sígueme para la Parte 4!

Milagros Espinoza Cunya